Team StockPulse Technical Report

Project 7: Real-Time Stock Market Monitor

1. Introduction

This project focuses on building a real-time stock monitoring application using C#. The system demonstrates concurrent programming through the Task Parallel Library (TPL) and data analysis using LINQ. Our goal is to efficiently fetch and process multiple stock data streams simultaneously while maintaining high performance and responsiveness.

In todays financial environments, stock prices fluctuate rapidly, often changing within milliseconds. Systems that rely on sequential data processing struggle to keep up with the speed and volume of these consistent updates. As a result, delayed or outdated data can lead to misinformed decision-making.

To address this challenge, this project implements asynchronous programming and parallel execution techniques to simulate a real-time stock monitoring system. By combining concurrency with structured data analysis, our application is able to provide timely insights and an accurate representation of market activity.

2. System Architecture

The application is structured into three main components:

- **Stock Model** – Represents stock data, including symbol, price, percentage change, and timestamp of the last update
- **TPL Layer** → Executes parallel stock fetch tasks
- **Stock Service** – Simulates API calls, real-time stock data retrieval, using asynchronous methods.
- **LINQ Layer** → Processes and analyzes data
- **Main Program** – Controls execution, manages concurrency, processes data, and displays results in a console interface

The system follows a simple but effective pipeline:

Data Fetching → Parallel Execution → Data Processing → Output Display

First, stock data is generated through the StockService using asynchronous calls. These calls are executed concurrently using TPL, allowing multiple stock requests to run at the same time. Once the data is collected, LINQ is applied to process and analyze the information. Finally, the processed data is displayed through a console-based user interface that updates continuously.

This design ensures separation of concerns, making the system easier to maintain, extend, and debug.

3. Technical Implementation

The application integrates several key technologies, primarily the Task Parallel Library (TPL) and LINQ

Task Parallel Library (TPL)

TPL is used to execute multiple stock data requests concurrently using Task.WhenAll(). Instead of fetching each stock one at a time, all requests are launched simultaneously, significantly reducing total execution time and eliminating bottlenecks.

LINQ (Language Integrated Query)

LINQ is used to efficiently process and analyze stock data. The following operations are implemented:

- Sorting stocks by performance using **OrderByDescending**
- Selecting the top gainer and top loser using **First**
- Filtering stocks with positive performance using **Where**
- Calculating the average stock price using **Average**

This combination enables a high-performance, real-time analytical pipeline.

Asynchronous Programming

The use of async and await ensures that operations are non-blocking. This allows the application to remain responsive while waiting for data to be retrieved.

Console User Interface

A dynamic console interface is used to display stock data in a structured format. The interface refreshes continuously, simulating a live data feed.

Additionally, the system simulates real-world API latency using Task.Delay(), with delays ranging from 500 to 1500 milliseconds. This helps demonstrate how the system would behave under real network conditions.

4. Implementation Overview

4.1 Stock Model

Represents each stock entity in the system:

```
public class Stock
{
  public string Symbol { get; set; }
  public decimal Price { get; set; }
  public decimal ChangePercent { get; set; }
  public DateTime LastUpdated { get; set; }
}
```

4.2 Stock Data Simulation (Async Service)

Simulates real-time API behavior using asynchronous delay:

```
public async Task<Stock> FetchStockDataAsync(string symbol)
{
  await Task.Delay(random.Next(500, 1500));

  return new Stock
  {
    Symbol = symbol.ToUpper(),
    Price = Math.Round((decimal)(random.NextDouble() * 500 + 50), 2),
    ChangePercent = Math.Round((decimal)(random.NextDouble() * 10 - 5),
2),
    LastUpdated = DateTime.Now
  };
}
```

5. Task Parallel Library (TPL) Implementation

```
List<Task<Stock>> stockTasks = stockSymbols
  .Select(symbol => stockService.FetchStockDataAsync(symbol))
  .ToList();

Stock[] stocks = await Task.WhenAll(stockTasks);
```

Benefits:

- All 7 stock requests execute simultaneously
- Eliminates sequential waiting
- Reduces total fetch time to the slowest task (~1–1.5 seconds)

6. LINQ Data Analysis

6.1 Sorting Stocks

```
var sortedStocks = stocks
  .OrderByDescending(stock => stock.ChangePercent)
  .ToList();
```

6.2 Top Gainer and Loser Selection

```
var topGainer = stocks.OrderByDescending(s => s.ChangePercent).First();
var topLoser = stocks.OrderBy(s => s.ChangePercent).First();
```

6.3 Filtering Gainers

```
var gainers = stocks.Where(s => s.ChangePercent > 0).ToList();
```

- Identifies all stocks with positive growth
- Used to calculate market sentiment

6.4 Aggregation (Average Price)

```
decimal avgPrice = Math.Round(stocks.Average(s => s.Price), 2);
```

- Provides overall portfolio price insight
- Eliminates manual looping and summation

7. Console output features

The StockPulse application dashboard includes the following features:

- Real-time stock monitoring through a continuous refresh loop
- Concurrent data fetching for multiple stock symbols
- Sorting stocks based on percentage change (highest to lowest)
- Identification of the top gaining and top losing stocks
- Filtering of stocks that are currently gaining value
- Calculation of the average price across all monitored stocks
- Display of the number of stocks gaining versus total stocks
- Interactive user control to refresh data or exit the application

The system monitors multiple stock symbols, including AAPL, MSFT, TSLA, AMZN, GOOG, NVDA, and META. Each refresh cycle updates stock prices, percentage changes, and timestamps, giving the impression of a live financial dashboard.

Example:

Average Price: $284.63
Stocks Gaining: 5/7

8. Result and Performance Analysis

The application was tested by executing multiple stock data requests concurrently and comparing performance against a sequential approach.

Results showed:

- The system remained responsive even during continuous refresh cycles
- LINQ provided efficient and readable data processing
- Simulated network delays did not negatively impact performance due to parallel execution

- Improved performance compared to sequential execution

In a sequential model, fetching seven stocks with delays of up to 1.5 seconds each could take over 10 seconds in total. With TPL and parallel execution, all stock data is retrieved in approximately the time of the slowest individual request, typically between 1 and 1.5 seconds.

This demonstrates the effectiveness of concurrency in improving application performance and scalability.

9. Key Technologies Used

- C# (.NET)
- Task Parallel Library (TPL)
- LINQ (Language Integrated Query)
- Async/Await Programming
- Console Application UI

10. Team Collaboration

The project was developed using collaborative software development practices:

- GitHub was used for version control and code management
- Tasks were divided among team members, including development, testing, and documentation
- Regular team discussions were held to review progress and resolve issues
- Code integration was performed carefully to maintain consistency and functionality

This structured approach ensured efficient teamwork and successful project completion.

11. Conclusion

The StockPulse Real-Time Stock Market Monitor successfully demonstrates the application of parallel programming and data analysis techniques in a real-world scenario. By leveraging the Task Parallel Library (TPL) and LINQ, the system efficiently processes multiple streams of stock data and transforms them into meaningful insights.

The project highlights the importance of concurrency in modern application development and shows how structured data processing can improve both performance and readability. Overall, the application serves as a strong example of how real-time systems can be built using C# and .NET technologies.